

Laboratoire 7

Parallélisme des tâches

Départements : TIC
Unité d'enseignement HPC

Auteur : Urs Behrmann
Professeur : Alberto Dassatti
Assistant : Bruno Da Rocha Carvalho
Classe : HPC
Salle de labo : A09
Date : 19 mai 2026

Contents

1	Analyse des k-mers	3
1.1	Inefficacités du code fourni	3
1.2	Améliorations mono-threadées	3
1.3	Performances mono-threadées	4
1.3.1	Analyse <code>perf stat</code>	4
1.3.2	Analyse <code>perf record / perf report</code>	5
1.4	Stratégie de parallélisation	6
1.5	Comparaison mono-thread / multi-thread	6
2	Activité Pan-Tompkins	7
2.1	Partie parallélisée	7
2.2	Stratégie de parallélisation	7
2.3	Choix, intérêt et efficacité	7

1 Analyse des k-mers

Trois exécutables : `k-mer` (code fourni), `k-mer-st` (mono-thread optimisé), `k-mer-mt` (OpenMP). Mesures sur `inputs/large.txt` (≈ 5 M caractères), $k = 3$, compilation `-O2`. Le chronomètre (`clock_gettime`) couvre uniquement la boucle de comptage des k-mers, hors lecture du fichier.

1.1 Inefficacités du code fourni

Le programme initial (`main.c`) présente deux problèmes majeurs :

- **I/O répétées** : pour chaque position i , `read_kmer` ouvre le fichier (`fopen`), se repositionne (`fseek`), lit k octets (`fgetc`) puis ferme (`fclose`). Sur un fichier de 5 M caractères, cela représente environ 5 M ouvertures de fichier — un coût système dominant, totalement évitable.
- **Recherche linéaire** : `add_kmer` parcourt toute la table avec `strcmp` à chaque insertion. Avec beaucoup de k-mers distincts, la complexité approche $O(n \times m)$ (n = positions, m = entrées uniques), ce qui ralentit fortement le traitement même une fois les I/O supprimées.

1.2 Améliorations mono-threadées

La version `k-mer-st` (`main_st.c`) conserve la même table et la même interface, mais applique des optimisations déjà vues dans les labos précédents :

- **Lecture unique** : le fichier est chargé entièrement en mémoire (`fread` dans un tampon), comme l'évitement d'allocations répétées au labo 6.
- **Accès direct** : chaque k-mer est extrait par `memcpy` depuis le tampon, sans réouverture du fichier.
- **Compilation -O2** : comme au labo 4, le compilateur vectorise et optimise les boucles simples (`memcpy`, boucle principale).

La structure de données (tableau dynamique + recherche linéaire) reste volontairement inchangée pour isoler le gain lié aux I/O.

1.3 Performances mono-threadées

Sur `large.txt` avec $k = 3$:

Version	Temps (s)
<code>k-mer</code> (fourni)	170,8
<code>k-mer-st</code> (optimisé)	136,6

La version optimisée est environ **25 %** plus rapide ($170,8\text{s} \rightarrow 136,6\text{s}$). Le gain confirme que les I/O répétées étaient un goulot important, mais le temps reste élevé : la recherche linéaire dans `add_kmer` domine désormais le profil. Sur un petit fichier (`input.txt`, 7 caractères), la différence est négligeable car le coût des `fopen` n'a pas le temps de s'accumuler.

1.3.1 Analyse perf stat

Mesure sur `k-mer-st` avec `sudo perf stat -d` :

```
sudo perf stat -d ./k-mer-st ../inputs/large.txt 3
```

Compteur	Valeur	Interprétation
Temps écoulé	137,26 s	Cohérent avec le chronomètre interne (136,6 s)
Temps user / sys	137,21 s / 0,02 s	Quasi tout le temps en espace utilisateur
CPUs utilisés	1,000	Mono-thread confirmé
Instructions	$1,39 \times 10^{12}$	Volume de travail élevé
Cycles	$6,43 \times 10^{11}$	—
IPC	2,16	Bon débit d'instructions par cycle
Branches / miss	$2,63 \times 10^9$ / 0,81 %	Boucles prévisibles (<code>strcmp</code> , <code>memcpy</code>)
L1-dcache miss	25,21 %	Accès mémoire dispersés (table de k-mers)
LLC miss	0,02 %	La RAM reste rarement sollicitée

Le rapport user/sys (137,21 s / 0,02 s) montre que les I/O système ne dominent plus : contrairement au code fourni, le programme passe presque exclusivement dans `add_kmer` et la boucle principale. L'IPC de 2,16 indique que le CPU reste relativement bien utilisé malgré l'accès non contigu à la table de k-mers.

En revanche, **25 %** de misses L1 confirment que la table de k-mers n'est pas cache-friendly : chaque appel à `strcmp` parcourt des chaînes et des structures allouées dynamiquement (`realloc`), ce qui génère des accès mémoire non contigus. Les misses LLC restent faibles (0,02 %) : le goulot n'est pas la RAM mais la latence L1 et le volume d'instructions (1,39 T) produit par les millions de comparaisons de chaînes. Le faible taux de branch-misses (0,81 %) est cohérent avec des boucles `for` et `strcmp` dont le comportement est stable.

1.3.2 Analyse perf record / perf report

Profil par fonctions avec `perf record -g` :

```
sudo perf record -g ./k-mer-st ../inputs/large.txt 3
perf report
```

Fonction	Samples	Self
<code>__strcmp_avx2 (libc)</code>	84,93 %	77,30 %
<code>add_kmer</code>	15,02 %	14,98 %
<code>strcmp@plt</code>	7,51 %	7,50 %
<code>main</code>	0,03 %	0,01 %
<code>__memmove_avx_unaligned_erms</code>	0,03 %	0,03 %

Le profil confirme que **près de 85 %** du temps est passé dans `__strcmp_avx2`, la version vectorisée AVX2 de `strcmp` fournie par la libc. C'est la conséquence directe de la recherche linéaire dans `add_kmer` : pour chaque position du fichier, le programme compare le k-mer courant à toutes les entrées déjà stockées. La fonction `add_kmer` elle-même apparaît à 15 %, correspondant à la boucle de parcours de la table et aux appels à `realloc/strcpy` lors d'insertions.

À l'inverse, `memcpy` (`__memmove_avx_unaligned_erms`) ne pèse que 0,03 % : l'optimisation de lecture en tampon a bien éliminé les I/O comme goulot mesurable. `main` est quasi absent du profil (0,03 %), ce qui indique que la boucle de comptage — et non l'affichage des résultats — concentre le travail. Aucune fonction liée aux fichiers (`fopen`, `fread`, `fgetc`) n'apparaît dans le top du rapport, validant l'analyse du `perf stat` précédent.

1.4 Stratégie de parallélisation

La version `k-mer-mt` (`main_mp.c`) repose sur OpenMP :

- **Répartition du travail** : `#pragma omp for schedule(static)` distribue les positions $i \in [0, \text{taille} - k]$ entre les threads de façon équilibrée. Chaque thread traite un bloc contigu de positions.
- **Tables locales** : chaque thread possède sa propre `KmerTable` (`locals[tid]`). Aucun verrou pendant le comptage — les threads écrivent dans des structures disjointes, ce qui évite la contention.
- **Fusion séquentielle** : après la région parallèle, `merge_kmer_table` combine les tables locales dans l'ordre des threads (0, 1, ...). Cela préserve l'ordre de première apparition des k-mers, identique à la version séquentielle.
- **Cas limites** : si $k > \text{tailledufichier}$, le programme quitte avant la parallélisation (comme en séquentiel). Aucun chevauchement entre blocs n'est nécessaire : chaque position i est traitée par exactement un thread ; les k-mers à la frontière de deux blocs sont comptés intégralement par le thread propriétaire de cette position.
- **Lecture du fichier** : effectuée une seule fois avant `#pragma omp parallel`, partagée en lecture seule par tous les threads.

1.5 Comparaison mono-thread / multi-thread

Version	Temps (s)	Accélération vs <code>k-mer-st</code>
<code>k-mer</code>	170,8	0,80×
<code>k-mer-st</code>	136,6	1,00×
<code>k-mer-mt</code>	64,7	2,11×

La version multi-threadée divise le temps par environ **2,1** par rapport au mono-thread optimisé, et par **2,6** par rapport au code fourni. L'accélération est inférieure au nombre de cœurs disponibles : le goulot d'étranglement principal reste `add_kmer` (recherche `strcmp` linéaire), exécutée en parallèle mais avec une complexité élevée par thread. La phase de fusion séquentielle en fin de traitement ajoute aussi un coût, proportionnel au nombre de k-mers distincts par thread.

Scalabilité : sur un petit fichier, le surcoût OpenMP (création de threads, tables locales, fusion) peut rendre `k-mer-mt` plus lent que `k-mer-st`. Sur `large.txt`, le volume de travail amortit ce surcoût et le parallélisme devient rentable.

2 Activité Pan-Tompkins

2.1 Partie parallélisée

Seule la **boucle de traitement des paquets** est parallélisée dans `ecg_stream_mp.c`. Pour chaque paquet, le travail comprend :

1. copie des échantillons dans un tampon local (`memcpy`) ;
2. appel à `ecg_analyze` (filtre, dérivée, intégration, seuil adaptatif, détection des pics R) ;
3. filtrage des pics selon la zone utile et conversion en indices globaux.

Non parallélisé : lecture du CSV, fusion des pics de tous les paquets, calcul des intervalles RR et écriture JSON. Ces étapes sont rapides ou doivent rester séquentielles pour préserver l'ordre des pics.

2.2 Stratégie de parallélisation

La version `ecg-mt` utilise OpenMP :

- **Répartition** : `#pragma omp for schedule(static)` sur l'indice de paquet k . Chaque paquet est indépendant une fois le signal chargé en mémoire.
- **Contexte par thread** : un `ECG_Context` par thread (`ctxs[tid]`), car `ecg_analyze` réutilise des tampons internes (`work`, `threshold`, etc.) non partageables.
- **Résultats locaux** : chaque paquet écrit dans `packet_peaks[k]`, sans verrou.
- **Fusion séquentielle** : après la région parallèle, les pics sont concaténés dans l'ordre des paquets ($k = 0, 1, \dots$). Les zones utiles étant disjointes, aucun pic n'apparaît deux fois.
- **Cas limites** : si le signal tient dans un seul paquet (≤ 1000 échantillons), la parallélisation n'apporte rien (un seul travail). Le dernier paquet peut être plus court que 1000 échantillons ; `ecg_analyze` gère une longueur variable.

2.3 Choix, intérêt et efficacité

Pourquoi paralléliser par paquet ? Chaque appel à `ecg_analyze` sur 1000 échantillons est une tâche autonome (filtres, intégration, détection). Le découpage en flux rend ce grain naturel pour OpenMP, contrairement à paralléliser l'intérieur de `ecg_analyze` qui modifierait le code du labo 1.

Pourquoi le chevauchement ? L'intégration de Pan-Tompkins utilise une fenêtre glissante ($\approx 300\text{ ms}$ de signal). Sans chevauchement, un pic R proche d'une coupure de paquet pourrait être manqué ou mal seuillé. Les 250 échantillons (0,5 s à 500 Hz) offrent une marge suffisante.

Mesures sur `test_ecg.csv` (7500 échantillons, Lead II) :

Implémentation	Exécutable	Temps (s)	Pics R
Ancienne (labo 1, signal entier)	<code>ecg_analyze</code> global	0,00015	96
Nouvelle (flux + OpenMP)	<code>ecg-mt</code>	0,00005	96

Les deux approches détectent **96 pics R** : le découpage par paquets avec chevauchement produit le même résultat que l'analyse globale du labo 1.

Le nombre de threads est limité au nombre de paquets (`omp_set_num_threads(min(paquets, cœurs))`), soit 10 ici. La version MT est **3× plus rapide** que l'ancienne sur le même fichier (0,05 ms vs 0,15 ms). Sur un flux plus long (signal concaténé), l'avantage augmente :

Signal	Ancienne (s)	<code>ecg-mt</code> (s)	Gain
7500 échant. (1×)	0,00015	0,00005	3×
75000 échant. (10×)	0,0015	0,00022	7×
750000 échant. (100×)	0,019	0,0023	8×

Évaluation : la parallélisation par paquet est pertinente dès que le nombre de threads est adapté au nombre de paquets. Répéter un petit signal ne change pas le rapport ; en revanche, un **flux continu réel** (plus long) ou un réglage correct des threads suffit à dépasser l'ancienne implémentation.

Utilisation des outils IA

Claude sonnet a été utilisé pour améliorer la formulation et corriger les fautes d'orthographe et de grammaire. Les choix et modifications du code sont les miennes.